Neural Language Modeling

Dr. Satadisha Saha Bhowmick

Preceptor

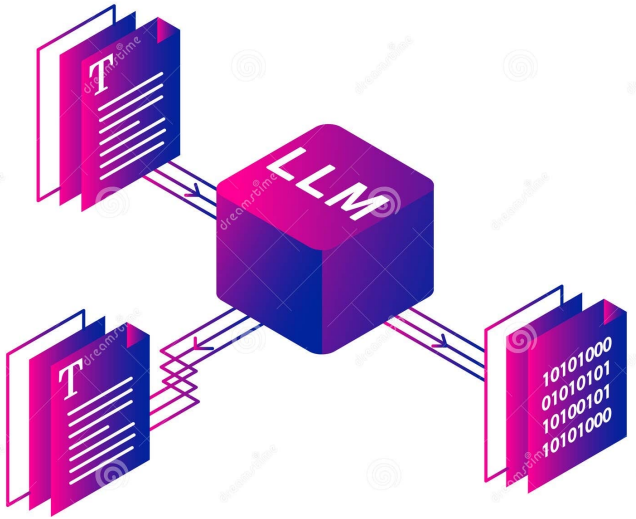
THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Objectives

- Motivation
- Feed Forward Neural Language Model
- Problem Statement
- Forward Inference
- Training a Neural Language Model

Motivation

- Statistical Language modeling aims to learn the joint probability distribution of sequences of words in a language/corpus.
- **Curse of Dimensionality** for n-gram models: if one wants to model the joint distribution of 10 consecutive words in a natural language with a vocabulary V of size 100,000, there are potentially $100000^{10} - 1 = 10^{50} - 1$ free parameters to be learnt.
  Training on any limited corpus cannot replicate these many combinations despite exhibiting the same vocabulary size.
- Need a model that simultaneously allow
  - Learning distributed representations for each word of dimension $m \ll |V|$
  - The probability function for word sequences, expressed in terms of these representations
- Generalization is obtained because a sequence of words not in the training set gets high probability if it is made of words that are representationally similar to words forming an already seen sentence



Fighting Curse of Dimensionality

Proposed approach of distributed representation learning of words

- associate with each word in the vocabulary a distributed word feature vector (a real valued vector in the multidimensional space $\mathbb{R}^m$)
- express the joint probability function of word sequences in terms of the feature vectors of these words in the sequence
- learn simultaneously the word feature vectors and the parameters of that probability function.

The number of feature vectors for the vectorial representation is of size 100, where for the vocabulary we often have a size of ~100000.

We will express the probability function as a product of conditional probabilities of the next words given the previous ones by simplifying/limiting the dependence assumption.

On a training set of text documents, we apply an optimization method such as gradient descent to **minimize the negative log-likelihood**.

Train a feed-forward neural network for this learning process.

- Pros: can handle larger histories than n-gram language model and better generalize context over similar words, more accurate at-word prediction.
- Cons: More complex and slower to train, especially for smaller tasks

Problem: Given a sentence in training
The cat is walking in the room
Can you generate comparable probability for
The dog is running in the room

A Neural Probabilistic Language Model

- A feedforward neural language model (LM) is a feedforward network that takes as input at time *t* a representation of some number of previous words (w_{t-1},w_{t-2}, etc.) and outputs a probability distribution over possible next words.
- **Simplifying assumptions**: Like the n-gram LM—the feedforward neural LM approximates the probability of a word given the entire prior context $P(w_t|w_{1:t-1})$ by approximating based on the $N-1$ previous words: $P(w_t|w_1,\dots,w_{t-1}) \approx P(w_t|w_{t-(N-1)},\dots,w_{t-1})$
- The overall task is to learn a function $f \geq 0$ such that $f(w_t,\dots,w_{t-(N-1)}) = P(w_t|w_{t-1},\dots,w_{t-(N-1)})$
- Since the predicted word is drawn from a closed vocabulary V, a constraint on the model is: $\sum_{i=1}^{|V|} f(i,w_{t-1},\dots,w_{t-(N-1)}) = 1$

Example: Using a 4-gram example, a neural net estimates the following probability $P(w_t = i|w_{t-3},w_{t-2},w_{t-1})$, where i is the i-th word of the vocabulary V.
Neural language models represent words in this prior context by their embeddings.
Using embeddings allows language models to generalize better unseen context (because comparable vectors are reflected in similarity!)

Suppose we've seen in training: I have to make sure the cat gets fed.
If our test set needs to predict the next word to: I forgot to make sure the dog gets

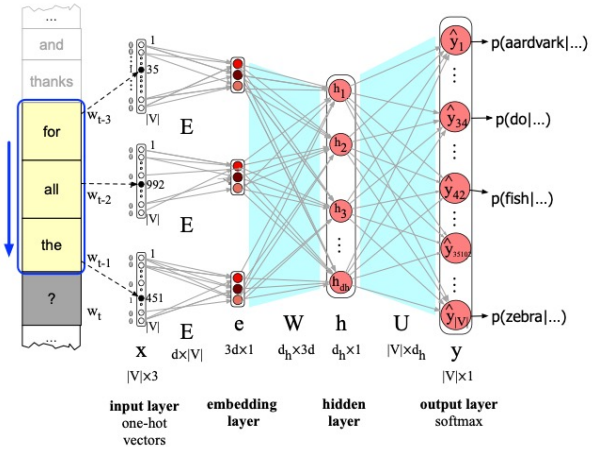
N-gram LM cannot predict '*fed*' if it was never seen after the word '*dog*' in training.
A neural LM can ensure '*cat*' and '*dog*' to have similar embeddings. So, it will be able to generalize from the "*cat*" context to assign a high enough probability to "*fed*" even after seeing "*dog*".

Forward Inference in Neural LM

Given an input context (previous words), forward Inference or Decoding for neural LM is the task of executing a forward pass on the network to produce a probability distribution over possible outputs that represent the next words.

- Represent each of the N previous words as a one-hot vector of length $|V|$, one dimension for each word in the vocabulary. For example, if one of the word in the context is v_5, the 5th word in V the one-hot vector is `[0 0 0 0 1 0 0 ... 0 0 0 0]` / `1 2 3 4 5 6 7 ... ... |V|`

  If N=4, then $w_{t-1}, w_{t-2}, w_{t-3}$ each have their own one-hot vectors.
- Multiply the one-hot vectors by an embedding weight matrix E, where each word is its own column vector of d dimension – representing its corresponding embedding. E has dimensionality of $d \times |V|$. Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects out the relevant embedding column vector for word i
- Embedding vectors of all context words are concatenated to produce the embedding layer $\boldsymbol{e}$
- Multiply by weight W and add bias b; pass through the ReLU activation function to get the hidden layer $\boldsymbol{h}$
- Multiply by $\boldsymbol{U}$ to project real valued estimates on the output layer consisting of $|V|$ nodes
- Apply softmax to convert to probabilities: $\hat{P}(w_t = i | w_{t-1}, \ldots, w_{t-(N-1)}) = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}$



Forward inference in a feed-forward LM with window size 3

$$\mathbf{e} = [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}]$$
$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{e} + \mathbf{b})$$
$$\mathbf{z} = \mathbf{U}\mathbf{h}$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

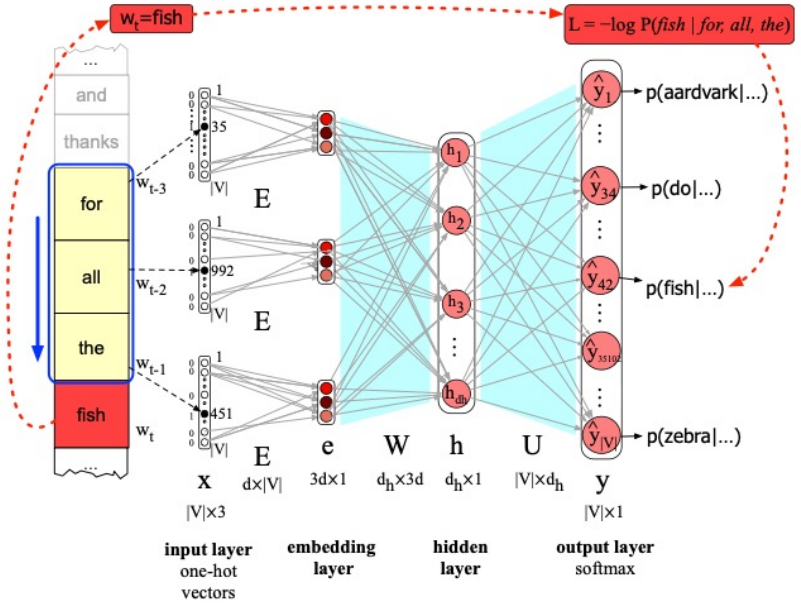
Training the Neural Language Model

Self Training or **Self Supervision** like Word2Vec.

- We take a corpus of text as training material and at each time step t ask the model to predict the next word.
- Train the model to minimize the error in predicting the true next word in the training sequence. No gold standard labels needed since words are already available in corpus!
- Training involves setting the parameters $\boldsymbol{\theta} = \boldsymbol{E}, \boldsymbol{W}, \boldsymbol{U}, \boldsymbol{b}$. For some tasks it would be okay to freeze E (held constant while training the rest of the network) with initial word2vec values. But here we want to learn them ofcourse!
- Loss Function is negative log likelihood.

  $L_{CE} = -\log P(w_t|w_{t-1}, \ldots, w_{t-(N-1)})$

  Apply Gradient Descent using Backpropagation all the way back to the Embedding layer. Training thus not only sets the weights W and U of the network, but also as we're predicting upcoming words, we're learning the embeddings E for each word that best predict upcoming words.



Training the feed-forward LM